DressMe - Code du chat BO5

Le code du chat BO5 se trouve principalement dans **le dossier dressme**. Le dossier bo5 contient surtout le notebook d'origine, l'adaptateur entraîné et les résultats d'entraînement.

Projet : C:/Users/GIGABYTE/Desktop/DressMe. Les chemins ci-dessous sont relatifs à cette racine.

1. Le fonctionnement général du chat

Fichier	Sa fonction
api.py	Point d'entrée de l'application FastAPI. Branche les routes du chat, vérifie l'authentification et transmet l'identité et le genre du compte au chat.
dressme/routes.py	Expose les API /api/assistant/… : envoyer un message, récupérer l'historique, enregistrer les préférences, gérer le dressing et afficher les images.
dressme/engine.py	Le cœur de BO5. Coordonne la compréhension du message, la recherche, les contraintes, l'assemblage des tenues, BO1, le classement et la mémoire de conversation.

Dans **engine.py**, les méthodes principales sont :

Méthode	Ce qu'elle fait
handle()	Traite un message complet et choisit l'action : composer, chercher des articles, remplacer, comparer, expliquer ou enregistrer une préférence.
retrieval()	Initialise le RAG et recherche les articles correspondant à la demande.
search()	Calcule les signaux de recherche et les calibre parmi les articles admissibles.
_assemble()	Construit des combinaisons de vêtements, applique les contraintes et demande leurs scores à BO1.
_retain()	Conserve un ensemble limité de candidats en tenant compte de la pertinence et des préférences.
rank()	Classe les tenues selon BO1, la pertinence et les préférences.
_public()	Prépare les résultats et les explications pour leur affichage dans l'interface.

Le remplacement d'une pièce et le What-if sont gérés dans handle(). Le remplacement conserve les autres pièces et modifie la tenue sélectionnée. Le What-if construit ou reprend les scénarios et calcule leurs différences.

2. La compréhension des demandes : le LLM et le parseur

Fichier	Sa fonction
dressme/parser.py	Charge Qwen3-1.7B avec ton adaptateur LoRA. Transforme le message en plan JSON : action, critères, nombre de tenues, tenue visée et pièce à remplacer. Vérifie aussi la cohérence du modèle et de l'adaptateur.
dressme/schemas.py	Définit les structures Pydantic : Facet pour un critère, Turn pour la demande comprise et ComposeSpec pour la recherche à exécuter. Refuse les champs et valeurs non autorisés.
dressme/grounding.py	Contrôle et corrige certaines sorties du parseur à partir des mots réellement présents dans le message : quantité, couleurs, vêtements, remplacement et référence à la deuxième tenue, par exemple.
dressme/constraints.py	Transforme le plan en critères exécutables et vérifie les articles : genre, couleur, type, budget, exclusions, dressing, etc. Produit les rapports indiquant quels critères passent ou échouent.
dressme/context.py	Gère les contextes comme sport, dîner, bureau, cérémonie, plage ou randonnée. Utilise les descriptions des articles pour écarter certaines pièces incompatibles avec l'usage demandé.

Par exemple, pour **« Remplace les chaussures de la deuxième tenue par des baskets blanches »** :

• **parser.py** extrait l'action, le numéro de tenue, les chaussures et la couleur.

• **grounding.py** vérifie ces informations dans le message.

• **schemas.py** valide la structure.

• **engine.py** conserve les autres pièces de la deuxième tenue et recherche son remplacement.

3. Le RAG, le catalogue et les recommandations

Fichier	Sa fonction
dressme/catalog.py	Charge le catalogue BO1 et construit les fiches produits : identifiant, description, rôle, couleurs, genre, image et provenance des informations. Vérifie l'alignement des données.
dressme/rag.py	Réalise la recherche hybride : BM25 pour les mots et multilingual-e5-small pour la proximité sémantique. Découpe les descriptions en passages, calcule les embeddings et conserve l'index en cache.
dressme/preferences.py	Mesure le respect des préférences, par exemple une préférence pour une couleur ou un style. Fournit les détails de leur contribution au classement.
dressme/selection.py	Calibre les scores de recherche par rôle et sélectionne des propositions diversifiées. Tient compte des tenues récemment montrées pour limiter les répétitions.
dressme/bo1.py	Le pont entre BO5 et BO1. Charge le vrai modèle exporté OutfitScorer, récupère les embeddings des articles et calcule la compatibilité des tenues.
bo1/score/dressme_core.py	Contient notamment l'architecture PyTorch OutfitScorer utilisée par ce pont BO1.

Le classement utilise par défaut **65 % de score BO1, 25 % de pertinence et 10 % de préférences**. Ces poids sont configurables dans engine.py.

BO5 assemble les combinaisons et BO1 les évalue. **Le chat n'utilise pas le MiniGPT de complétion pour générer ses tenues** ; cette fonctionnalité possède un endpoint séparé.

4. La mémoire et le genre du compte

Fichier	Sa fonction
dressme/state.py	Enregistre dans SQLite l'historique, les préférences, le dressing, les tenues affichées et les tenues récentes. Les données sont séparées par compte dans l'application intégrée.
dressme/identity.py	Applique automatiquement le genre du compte aux recherches. Un genre explicitement demandé dans le message reste prioritaire.
dressme/config.py	Centralise les chemins des modèles, du cache et des sessions ainsi que les paramètres de téléchargement et d'authentification Hugging Face.

Le genre vient du module utilisateur :

Fichier	Sa fonction
app/models.py	Stockage du champ gender.
app/auth/schemas.py	Validation des valeurs homme/femme.
static/studio/auth.js	Choix dans le formulaire d'inscription.

5. L'interface du chat

Fichier	Sa fonction
static/index.html	Contient la section du chat : messages, zone de saisie, boutons, préférences, dressing et diagnostic.
static/assistant.js	Envoie les demandes au serveur et affiche les réponses, les vêtements, les scores, les comparaisons et les erreurs. Recharge aussi l'historique après connexion.
static/assistant.css	Définit l'apparence du chat et des cartes de vêtements.

6. Les modèles et le travail d'entraînement

Emplacement	Contenu et fonction
bo5/bo5-integ (1).ipynb	Notebook d'origine de BO5. Le serveur exécute les modules Python extraits et adaptés dans dressme/.
bo5/parser_for_local/parser_adapter/	Ton adaptateur LoRA, son tokenizer, sa configuration et son reçu d'entraînement.
bo5/resume_latest/	Checkpoints conservés pour la reprise ou l'analyse de l'entraînement. Le chat charge l'adaptateur exporté dans parser_adapter.
.cache/models/	Poids locaux de Qwen et, selon l'installation, d'E5.
bo1/score/	Modèle BO1, embeddings, catalogue et images utilisés pour scorer les tenues.
bo5/parser_for_local/reports/	Rapports d'entraînement, évaluations du parseur, benchmarks et courbes.

7. Les tests du chat

Fichier	Ce qu'il vérifie
tests/test_integration.py	Les composants intégrés et leurs contrats.
tests/test_conversation.py	Les échanges successifs, les références aux tenues et les modifications.
tests/test_user_chat_integration.py	Le lien entre compte utilisateur et chat, notamment le genre et l'isolation des données.
tests/test_relevance.py	La pertinence selon les contextes et la diversité des propositions.
scripts/verify_relevance.py	Les scénarios de vérification avec les vrais modèles.
scripts/verify_conversation.py	Les scénarios de conversation avec les vrais modèles.
scripts/verify_count_parser.py	Les demandes portant sur le nombre de tenues et les corrections du parseur.

Le parcours d'une demande est donc :

Interface → API → Qwen/LoRA → validation → RAG et filtres → assemblage BO5 → score BO1 → classement et diversité → affichage et sauvegarde.

Les responsabilités que tu peux présenter comme tes « agents » sont réparties entre ces modules. Dans le code actuel, **un seul LLM est chargé pour comprendre les demandes** ; la recherche, le scoring, la sélection et la rédaction des explications sont assurés par les autres composants.